Orange | Founding AI Engineer — Design Note

Scope: fully local home-service concierge; mock providers and payments. **Evidence baseline:** G4 source `b6d4bb72e5108e691fe38d1dfc994cf7dae8771c`; final G4 release `artifacts/receipts/g4_release_20261103e/receipt.json`. This document is intentionally limited to approximately two printed pages.

Thesis and user journey

A model should **understand the request**, not possess authority to spend, book or erase records. The user describes a task conversationally; the system obtains real mock-provider availability and an exact hold quote; the user reviews an immutable, session-bound approval card; a durable worker executes the fixed provider API; and the app reports success **only after authoritative readback**. Booking, rescheduling and cancellation use the same consent discipline. A browser presents conversation and live sanitized trace side by side.

Boundary: browser → two authenticated HTTP API replicas → PostgreSQL workflow/approval/operation/outbox authority → two leased workers → isolated mock-provider process/ledger. A Groq `openai/gpt-oss-20b` adapter provides schema-constrained intent *proposals*; a deterministic offline interpreter backs reproducible evaluations. Model output cannot call write tools, approve itself, assert provider success or overwrite identity. The employer's eight tool signatures are unchanged.

Correctness before fluency

Consent: a fingerprint binds identity, workflow revision, provider, slot, hold ID, quoted amount and action. A changed provider/price/slot invalidates earlier approval; text such as “yes” is not a privileged confirmation. Browser clicks issue an explicit, authenticated, CSRF-checked confirmation.

Ambiguous provider outcomes: stable idempotency keys name logical operations. After a timeout, the worker reads `get_booking_status` and `list_bookings` before any additional attempt; it preserves a truthful `OUTCOME_UNKNOWN` when evidence is inconclusive. Persistent jobs use leases/fences, bounded attempts and recovery windows. Process death and response loss are regression-tested against provider SQL—not only UI messages.

Untrusted text and memory: provider reviews are displayed as untrusted and excluded from the Groq intent context. Semantic constraints are independently grounded in the latest request, including IST-relative dates, budgets and morning times. Memory is user-scoped, provenance/version-aware and explicitly correctable/forgettable; in-flight stale contexts are discarded on revision mismatch. Forget provides operational suppression/tombstones, **not** forensic deletion from backups.

Observability: runs have trace IDs, span IDs, workflow/operation linkage, model version, token usage where returned, bounded execution budgets, sanitized payload shapes and errors. Raw secrets, history, reviews and personal preference values are not recorded. An unmodified E12 trace additionally records the booking tool's hashed hold/key arguments without raw identifiers; all eight method shapes have explicit redaction tests. Exact provider and approval effects are independently reconciled with PostgreSQL.

Evidence, failure and tradeoffs

A clean source checkout passed **65 original SQLite tests + 39 distributed PostgreSQL/process tests + 28 offline end-to-end scenarios + 20 model-adapter defenses**, with no skips. The separate G4 live integration yielded three model responses plus two browser responses; five model spans were reconciled. A real Chromium path exercised quote correction, explicit approval, timeout recovery, reconnect, memory forget, reschedule, cancel and logout. Three representative red→fix→regression defects are documented in `THREE_BUGS.md`. **Raw timeout-after-success excerpt:** `book → provider_timeout / OUTCOME_UNKNOWN → VERIFYING → get_booking_status: booked → list_bookings: count=1 → BOOKED` (full unmodified worker run: `artifacts/traces/timeout_after_success.jsonl`; annotation: `TIMEOUT_TRACE_ANNOTATION.md`). These are **local scenario results**, not a broad model reliability estimate.

Tradeoff: PostgreSQL, separate provider authority and explicit work queues add setup cost over an in-memory chatbot but make retries, permission boundaries and recovery auditable. Two local replicas/workers are not production high availability; fixture auth is not OIDC/TLS.

How to reach 100× usage — an evidence-gated plan, not a claim

First establish the denominator. Prior local 100-active-workflow synthetic experiments showed full-journey p95 of approximately **190–264 seconds** with artificial faults and preparation batching—not 100× throughput. These measurements are not an SLO.

Likely first bottlenecks: PostgreSQL connection/lock contention, slow provider status recovery, and model rate-limit/latency variability—not model reasoning sophistication. At 10× demand: benchmark arrival rate, accepted bookings/minute, p50/p95/p99 end-to-end latency, tool budgets, lock waits, DB pool saturation, model rate-limit/invalid-schema incidence, queue age, unresolved outcome age and duplicate effects. Scale stateless HTTP replicas and partition queues by operation/tenant only after measuring; right-size pools against PostgreSQL limits.

At 100× demand: shard/partition operational ledgers by stable tenant/region keys, isolate provider rate budgets, use backpressure and a bounded dead-letter/reconciliation queue, aggregate traces with sampled diagnostics but unsampled authority audits, exercise crash/partition and replay testing. Maintain explicit per-tenant quotas, token/cost limits, OIDC/TLS and deletion/recovery policies. **Promotion gate:** zero unauthorized or duplicate effects across fault-injected load, bounded unknown outcome age, agreed p95/SLO and repeatable restore/replay—measured on production-like infrastructure before claiming scale.

Not claimed: real provider/payment integrations, generalized prompt-injection immunity, production HA, universal model understanding, 100× measured speedup or guaranteed hiring outcomes.